CRUD MASTER

Microservices API Infrastructure

Complete Step-by-Step Project Guide

Stack: Node.js | Express | PostgreSQL | RabbitMQ | Vagrant | PM2

Table of Contents

1. Project Overview
2. Architecture
3. Prerequisites & Installation
4. Project File Structure
5. Environment Variables (.env)
6. Step 1 — Build the Inventory API
7. Step 2 — Build the Billing API
8. Step 3 — Build the API Gateway
9. Step 4 — Writing the Vagrantfile
10. Step 5 — Writing the Setup Scripts
11. Step 6 — Using PM2
12. Step 7 — OpenAPI / Swagger Documentation
13. Step 8 — Postman Testing
14. Audit Checklist & Verification
15. Troubleshooting

1. Project Overview

CRUD Master is a microservices project that simulates a movie streaming platform. It consists of three independent services, each running inside its own Vagrant virtual machine (VM). The services communicate over HTTP and via a RabbitMQ message queue, demonstrating key principles of distributed system design.

The three services are:

- API Gateway — the entry point for all client requests, running on its own gateway-vm.
- Inventory API — a full CRUD REST API for managing movies, running on inventory-vm.
- Billing API — a message-driven service that processes orders from a RabbitMQ queue, running on billing-vm.

> **Note:** *The goal of this project is not only to build working APIs, but to understand how microservices communicate, how VMs are provisioned with Vagrant, and how to test resilience using message queues.*

---

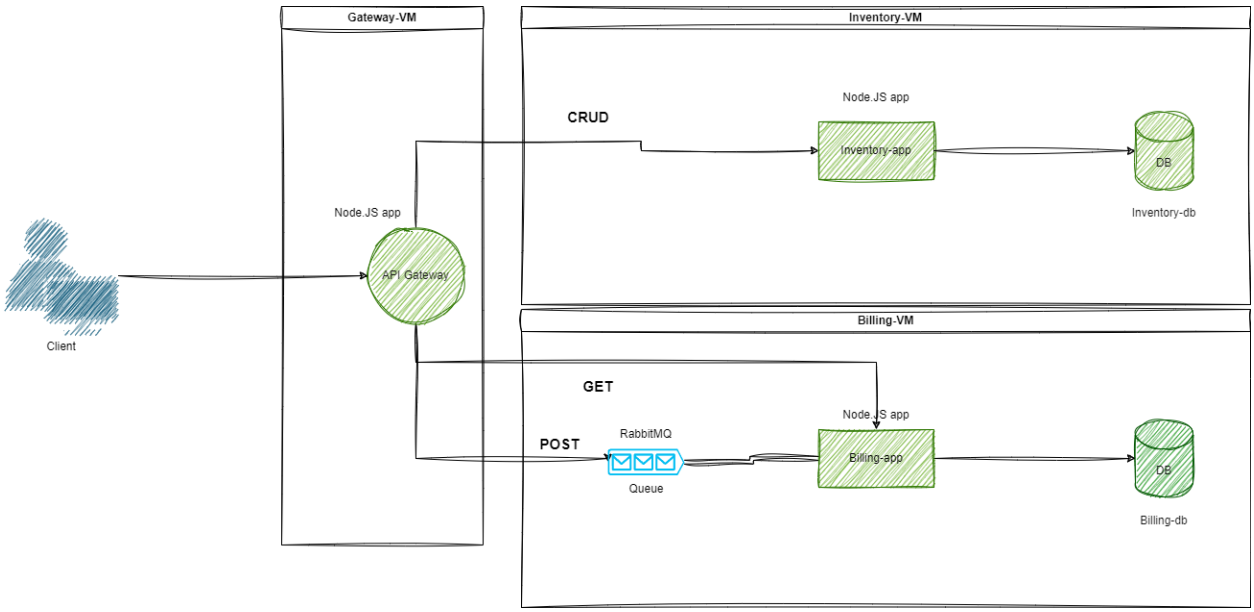
2. Architecture

The diagram below describes the flow of data through the system:

- A Client sends HTTP requests to the API Gateway.
- The API Gateway routes /api/movies/* requests to the Inventory API via HTTP proxy.
- The API Gateway routes /api/billing POST requests by publishing a message to RabbitMQ's billing_queue.
- The Billing API consumes messages from billing_queue and writes orders to the PostgreSQL orders database.
- If the Billing API is offline, messages remain in the queue and are processed when the API restarts (queue durability / resilience).

Communication protocols summary:

Route	Protocol	Destination
`/api/movies/*`	HTTP Proxy	Inventory API (inventory-vm)
`/api/billing POST`	RabbitMQ (AMQP)	Billing API (billing-vm)



3. Prerequisites & Installation

Before starting, ensure the following tools are installed on your HOST machine (your laptop/desktop, not inside a VM):

3.1 Required Tools

- VirtualBox (v6.1 or later) — Free VM hypervisor. Download from virtualbox.org
- Vagrant (v2.3 or later) — VM provisioning tool. Download from vagrantup.com
- Postman — API testing tool. Download from postman.com
- Node.js (v18 LTS or later) — For local development/testing before Vagrant
- Git — For version control

3.2 Installation Verification

Run these commands in your terminal to confirm installations:

```
virtualbox --help          # Should print VirtualBox CLI help
vagrant --version          # Should print: Vagrant 2.x.x
node --version             # Should print: v18.x.x or higher
npm --version              # Should print npm version
git --version              # Should print git version
```

 Important: On macOS with Apple Silicon (M1/M2), VirtualBox may not work. Use VMware Fusion or UTM as alternatives and update the Vagrantfile provider accordingly.

4. Project File Structure

Create the following directory and file structure from scratch. This is the complete layout for the project:

```
.
├── README.md
├── .env
├── Vagrantfile
├── scripts/
│   ├── setup_inventory.sh
│   ├── setup_billing.sh
│   └── setup_gateway.sh
└── srcs/
    ├── api-gateway/
    │   ├── package.json
    │   ├── server.js
    │   ├── routes.js
    │   └── proxy.js
    ├── inventory-app/
    │   ├── package.json
    │   ├── server.js
    │   └── app/
    │       ├── config/db.js
    │       ├── models/movie.model.js
    │       ├── controllers/movie.controller.js
    │       └── routes/movie.routes.js
    └── billing-app/
        ├── package.json
        ├── server.js
        └── app/
            ├── config/db.js
            ├── models/order.model.js
            └── controllers/order.controller.js
```

 Note: *The node_modules/ directories and package-lock.json files are auto-generated. Do NOT commit them. Add node_modules to your .gitignore file.*

5. Environment Variables (.env)

The .env file lives at the root of the project and stores ALL credentials and configuration values. No credential should ever be hardcoded inside any JavaScript source file. The Vagrantfile will read this file and inject the values into each VM as environment variables.

5.1 Creating the .env File

Create a file named .env at the root of your project directory with the following keys (replace placeholder values as needed):

```
# Inventory Database (on inventory-vm)
INVENTORY_DB_NAME=movies
INVENTORY_DB_USER=inventory_user
INVENTORY_DB_PASSWORD=inventory_pass
INVENTORY_DB_HOST=localhost
INVENTORY_DB_PORT=5432

# Billing Database (on billing-vm)
BILLING_DB_NAME=orders
BILLING_DB_USER=billing_user
BILLING_DB_PASSWORD=billing_pass
BILLING_DB_HOST=localhost
BILLING_DB_PORT=5432

# RabbitMQ (on billing-vm)
RABBITMQ_USER=rabbitmq_user
RABBITMQ_PASSWORD=rabbitmq_pass
RABBITMQ_HOST=192.168.56.12
RABBITMQ_PORT=5672
RABBITMQ_QUEUE=billing_queue

# API Ports
INVENTORY_PORT=8080
BILLING_PORT=8081
GATEWAY_PORT=3000

# VM Static IPs
GATEWAY_IP=192.168.56.10
INVENTORY_IP=192.168.56.11
BILLING_IP=192.168.56.12
```

5.2 Why Include .env in the Repository?

Normally, .env files are added to .gitignore to prevent leaking credentials. However, for this project the subject explicitly requires the .env to be committed so the auditor can reproduce the setup. This is an exception for educational purposes only.

> **Important:** In real production projects, NEVER commit .env files to public repositories. Use secret managers like AWS Secrets Manager or HashiCorp Vault instead.

6. Step 1 — Build the Inventory API

The Inventory API is a standard RESTful CRUD API built with Express.js and Sequelize ORM, backed by a PostgreSQL database named 'movies'. It runs on port 8080.

6.1 Initialize the Project

Inside the srcs/inventory-app/ directory, run:

```
npm init -y
npm install express sequelize pg pg-hstore dotenv
```

- express — HTTP web framework
- sequelize — ORM for PostgreSQL (abstracts raw SQL)
- pg — PostgreSQL driver for Node.js
- pg-hstore — Handles serializing/deserializing JSON data for PostgreSQL
- dotenv — Loads environment variables from .env into process.env

6.2 Configure the Database Connection (app/config/db.js)

This file creates and exports a Sequelize instance. It reads all credentials from environment variables — never hardcode them. The instance specifies 'postgres' as the dialect and connects to the host/port/credentials defined in .env. Call sequelize.authenticate() to verify the connection on startup.

6.3 Define the Movie Model (app/models/movie.model.js)

Use Sequelize's define() or Model.init() to create the 'movies' table schema. The columns must be:

- id — INTEGER, auto-increment primary key (Sequelize handles this automatically with the 'id' convention)
- title — STRING, cannot be null
- description — TEXT, can be null

Call sequelize.sync() in your server startup to auto-create the table if it does not exist. This is safe for development.

6.4 Write the Controller (app/controllers/movie.controller.js)

The controller contains the logic for each endpoint. Each function receives the Express req and res objects. Using Sequelize, implement:

- findAll() — returns all movies, supports optional title filter via req.query.title using Sequelize's Op.like operator
- findOne({ where: { id } }) — returns a single movie by primary key
- create(data) — creates a new movie row from req.body
- update(data, { where: { id } }) — updates fields for a given id
- destroy({ where: { id } }) — deletes a single movie
- destroy({ where: {} }) — deletes all movies (no WHERE clause = all rows)

Return appropriate HTTP status codes: 200 for success, 404 if movie not found, 500 for server errors. Always respond with JSON.

6.5 Define Routes (app/routes/movie.routes.js)

Create an Express Router and map HTTP methods to controller functions:

Method	Endpoint	Description
GET	`/api/movies`	Get all movies (supports ?title= query)
POST	`/api/movies`	Create a new movie
DELETE	`/api/movies`	Delete all movies
GET	`/api/movies/:id`	Get one movie by ID
PUT	`/api/movies/:id`	Update one movie by ID
DELETE	`/api/movies/:id`	Delete one movie by ID

6.6 Write the Server Entry Point (server.js)

server.js is the main entry point. It should:

1. Require dotenv and call dotenv.config() as the very first line so process.env is populated before anything else is imported.
2. Import Express and create an app instance.
3. Add express.json() middleware so POST/PUT request bodies are parsed automatically.
4. Import your routes and mount them under the /api prefix.
5. Import and call the db config to sync Sequelize models (sequelize.sync()).
6. Call app.listen(process.env.INVENTORY_PORT) to start the server.

6.7 Local Testing (Before Vagrant)

To verify the API works before setting up VMs, you need a local PostgreSQL instance. Create the database manually with:

```
psql -U postgres
CREATE DATABASE movies;
CREATE USER inventory_user WITH PASSWORD 'inventory_pass';
GRANT ALL PRIVILEGES ON DATABASE movies TO inventory_user;
```

Then start the server locally:

```
node server.js
```

Test with curl or Postman: POST to http://localhost:8080/api/movies with a JSON body containing 'title' and 'description'.

---

7. Step 2 — Build the Billing API

The Billing API does NOT expose any HTTP endpoints. Instead, it listens on a RabbitMQ queue called 'billing_queue'. When a message arrives, it parses the JSON, creates a database record in the 'orders' PostgreSQL database, and acknowledges the message. A key requirement is resilience: messages must persist in the queue when the API is offline and be processed when it restarts.

7.1 Initialize the Project

Inside srcs/billing-app/:

```
npm init -y
npm install amqplib sequelize pg pg-hstore dotenv
```

- amqplib — AMQP 0-9-1 client for Node.js, the protocol used by RabbitMQ
- Other packages are the same as in the Inventory API

7.2 Configure the Database (app/config/db.js)

Same pattern as Inventory API, but using BILLING_DB_NAME, BILLING_DB_USER, BILLING_DB_PASSWORD from process.env.

7.3 Define the Order Model (app/models/order.model.js)

Create the 'orders' table with the following columns:

- id — INTEGER, auto-increment primary key
- user_id — STRING, cannot be null
- number_of_items — STRING, cannot be null
- total_amount — STRING, cannot be null

7.4 Write the Consumer Logic (app/controllers/order.controller.js)

This is the core of the Billing API. The consumer function must:

7. Connect to RabbitMQ using amqplib.connect() with the connection URL built from environment variables: amqp://USER:PASS@HOST:PORT
8. Create a channel from the connection using connection.createChannel()
9. Assert the queue exists using channel.assertQueue('billing_queue', { durable: true }). The durable: true flag ensures the queue survives a RabbitMQ restart.
10. Set channel.prefetch(1) — this tells RabbitMQ to only deliver one message at a time to this consumer, ensuring reliable processing.
11. Call channel.consume() with a callback. Inside the callback, parse msg.content.toString() using JSON.parse() to extract user_id, number_of_items, and total_amount.
12. Use Sequelize to create a new order record in the database.
13. After successful database write, call channel.ack(msg) to acknowledge the message. This removes it from the queue. If you do NOT ack, RabbitMQ will re-queue it.

 Important: If you use channel.ack(msg) BEFORE writing to the database and the write fails, the message is lost forever. Always ack AFTER a successful database operation.

7.5 Write the Server Entry Point (server.js)

The server.js for the Billing API is simpler than Inventory. It should:

14. Call dotenv.config() first.
15. Sync the Sequelize models (sequelize.sync()).
16. Call the RabbitMQ consumer function to start listening. Wrap this in a retry loop or use setTimeout to wait for RabbitMQ to be ready on startup inside the VM, as RabbitMQ may take a few seconds to initialize after the VM boots.

 Note: *There is no app.listen() call in the Billing API since it does not serve HTTP requests. It simply connects to RabbitMQ and waits for messages.*

---

8. Step 3 — Build the API Gateway

The API Gateway is the single entry point for all external traffic. It proxies Inventory requests over HTTP and publishes Billing requests to RabbitMQ. It runs on port 3000.

8.1 Initialize the Project

Inside srcs/api-gateway/:

```
npm install express http-proxy-middleware amqplib dotenv
```

- http-proxy-middleware — Middleware to forward HTTP requests to another server

- amqplib — Same AMQP client as used in Billing API

8.2 Set Up the HTTP Proxy (proxy.js)

Create a proxy for all /api/movies/* routes. Using http-proxy-middleware's createProxyMiddleware(), configure the target to be the Inventory API's IP and port. The key settings are:

- target — set to http://INVENTORY_IP:INVENTORY_PORT using env vars
- changeOrigin: true — rewrites the Host header to match the target
- on.error — add an error handler to return 502 if the Inventory API is unreachable

Export the configured proxy middleware function so routes.js can use it.

8.3 Set Up RabbitMQ Publisher (routes.js)

The routes file handles POST /api/billing. The logic must:

17. Connect to RabbitMQ at the billing-vm's IP using the same amqplib.connect() pattern.
18. Assert the billing_queue exists with durable: true. This is critical — the gateway must be able to publish to the queue even when the Billing API consumer is offline.
19. Use channel.sendToQueue('billing_queue', Buffer.from(JSON.stringify(req.body)), { persistent: true }). The persistent: true flag tells RabbitMQ to save the message to disk, so it survives a RabbitMQ restart.
20. Return HTTP 200 to the client immediately after publishing. You do not wait for the Billing API to process it. This is what allows the gateway to return 200 even when billing-app is stopped.

 Note: *The RabbitMQ connection in the gateway can be established once at startup and reused, or created per request. A persistent connection at startup is more efficient.*

8.4 Write the Server Entry Point (server.js)

The gateway server.js should:

21. Call dotenv.config().
22. Create an Express app and add express.json() middleware.
23. Import routes from routes.js and apply them.
24. Import the proxy from proxy.js and mount it: app.use('/api/movies', proxy).
25. Start listening on GATEWAY_PORT.

8.5 Local Testing of the Gateway

With all three services running locally (node server.js in each directory), test the gateway at http://localhost:3000. A POST to /api/movies should appear in the movies database, and a POST to /api/billing should enqueue a message processed by the billing service.

---

9. Step 4 — Writing the Vagrantfile

The Vagrantfile is a Ruby-syntax configuration file that defines all three VMs. It lives at the root of the project. Vagrant reads it when you run vagrant up.

9.1 Loading Environment Variables

At the top of the Vagrantfile (before the Vagrant.configure block), load the .env file using Ruby's built-in file reading. Parse each line that is not a comment, split on '=', and store key-value pairs in a Ruby hash. Then use ENV['KEY'] = value to inject them into the Vagrant environment so they can be passed to shell provisioners.

9.2 VM Definitions

Define each VM inside a Vagrant.configure('2') do |config| block using config.vm.define. Each VM needs:

- A unique name (gateway-vm, inventory-vm, billing-vm)
- A base box — use ubuntu/focal64 (Ubuntu 20.04 LTS) for reliability
- A static private IP — assign IPs from your .env (e.g., 192.168.56.10, .11, .12). Use vm.network 'private_network', ip: ENV['GATEWAY_IP']
- A synced folder — sync srcs/ from the host into the VM: vm.synced_folder './srcs', '/home/vagrant/srcs'
- A shell provisioner — vm.provision 'shell', path: 'scripts/setup_gateway.sh', env: { ... } passing all necessary env vars

9.3 Passing Environment Variables to Provisioners

In the provision block, pass the relevant environment variables as a hash to the env option. For example, the billing VM provisioner needs all BILLING_DB_*, RABBITMQ_*, and BILLING_PORT variables. The inventory VM needs INVENTORY_DB_* and INVENTORY_PORT. The gateway VM needs all the IPs and ports it proxies to.

9.4 Key Vagrant Commands

```
vagrant up                  # Boot and provision all 3 VMs
vagrant status              # Show running status of all VMs
vagrant ssh gateway-vm      # SSH into the gateway VM
```

```
vagrant ssh inventory-vm    # SSH into the inventory VM
vagrant ssh billing-vm      # SSH into the billing VM
vagrant halt                # Gracefully shut down all VMs
vagrant destroy -f          # Delete all VMs (destructive!)
vagrant provision           # Re-run provisioning scripts
vagrant reload              # Restart VMs and re-apply config
```

 Note: *The first vagrant up will download the ubuntu/focal64 box (~500MB). This only happens once and is cached locally for future use.*

---

10. Step 5 — Writing the Setup Scripts

Each VM has a corresponding bash setup script in the scripts/ directory. These scripts run during vagrant up provisioning and install all required software, set up databases, configure services, and start the Node.js application with PM2.

10.1 scripts/setup_inventory.sh

This script runs on inventory-vm. It must perform these steps in order:

26. Update apt package index: apt-get update -y
27. Install Node.js: Use the NodeSource setup script for Node.js 18 LTS (curl -fsSL https://deb.nodesource.com/setup_18.x | bash - then apt-get install -y nodejs)
28. Install PostgreSQL: apt-get install -y postgresql postgresql-contrib
29. Start and enable PostgreSQL service: systemctl start postgresql && systemctl enable postgresql
30. Create the PostgreSQL user and database using sudo -u postgres psql -c commands. Create the user with the password from the INVENTORY_DB_PASSWORD env var. Create the movies database. Grant all privileges to the user.
31. Install PM2 globally: npm install -g pm2
32. Navigate to the synced app directory: cd /home/vagrant/srcs/inventory-app
33. Install Node.js dependencies: npm install
34. Write environment variables to a .env file inside the app directory using echo statements. This ensures the app can read its config with dotenv.
35. Start the application with PM2: pm2 start server.js --name inventory_app
36. Save the PM2 process list and configure PM2 to start on reboot: pm2 save && pm2 startup

10.2 scripts/setup_billing.sh

This script runs on billing-vm. In addition to Node.js, PostgreSQL, and PM2 (same as above), it must also install and configure RabbitMQ:

37. Update packages and install Node.js 18 (same as above)
38. Install PostgreSQL and create the orders database and billing user
39. Install RabbitMQ: apt-get install -y rabbitmq-server
40. Start and enable RabbitMQ: systemctl start rabbitmq-server && systemctl enable rabbitmq-server
41. Create a RabbitMQ user with the credentials from env vars: rabbitmqctl add_user $RABBITMQ_USER $RABBITMQ_PASSWORD
42. Set the user as administrator: rabbitmqctl set_user_tags $RABBITMQ_USER administrator
43. Grant permissions to the user: rabbitmqctl set_permissions -p / $RABBITMQ_USER '.*' '.*' '.*'
44. Optionally delete the default 'guest' user for security: rabbitmqctl delete_user guest
45. Install PM2 globally
46. Navigate to /home/vagrant/srcs/billing-app, run npm install
47. Write the .env file and start with PM2: pm2 start server.js --name billing_app
48. Save PM2 process list

 Important: RabbitMQ listens on port 5672 by default. The billing-vm's static IP (e.g., 192.168.56.12) must match RABBITMQ_HOST in your .env file so the gateway can reach it.

10.3 scripts/setup_gateway.sh

This script runs on gateway-vm. It is the simplest of the three:

49. Install Node.js 18
50. Install PM2 globally
51. Navigate to /home/vagrant/srcs/api-gateway
52. Run npm install
53. Write the .env file with INVENTORY_IP, INVENTORY_PORT, RABBITMQ_HOST, RABBITMQ_PORT, RABBITMQ_USER, RABBITMQ_PASSWORD, RABBITMQ_QUEUE, GATEWAY_PORT
54. Start with PM2: pm2 start server.js --name api_gateway
55. Save PM2 process list

 Note: *The gateway does NOT need PostgreSQL. It only needs Node.js, the app dependencies, and access to the other VMs over the private network.*

---

11. Step 6 — Using PM2

PM2 (Process Manager 2) is a production-grade process manager for Node.js. It keeps your apps alive, restarts them on crashes, and provides logging and monitoring. It is critical for this project because it allows you to start and stop individual services during audit testing.

11.1 Core PM2 Commands

```
sudo pm2 list                     # Show all managed apps and their status
sudo pm2 start <name_or_script>   # Start a stopped app
sudo pm2 stop <app_name>          # Stop a running app
sudo pm2 restart <app_name>       # Restart a running app
sudo pm2 delete <app_name>        # Remove app from PM2 list
sudo pm2 logs <app_name>          # Stream live logs
sudo pm2 logs <app_name> --lines 50  # Show last 50 log lines
sudo pm2 monit                    # Interactive monitoring dashboard
```

11.2 How PM2 Enables the Resilience Test

The audit requires demonstrating that messages sent to the billing queue while billing_app is stopped are processed when it restarts. Here is the exact test sequence:

56. SSH into billing-vm: vagrant ssh billing-vm
57. Stop the billing app: sudo pm2 stop billing_app
58. Confirm it is stopped: sudo pm2 list (status should show 'stopped')
59. From Postman, POST to /api/billing — you should get 200 OK from the gateway even though billing_app is down
60. Verify the message is NOT yet in the orders database: sudo -i -u postgres psql -c 'SELECT * FROM orders;' orders
61. Restart the app: sudo pm2 start billing_app
62. Check the database again — the order should now appear (RabbitMQ delivered the queued message)

 Note: *This works because RabbitMQ persists durable messages on disk. When billing_app reconnects and calls channel.consume(), RabbitMQ delivers all unacknowledged messages.*

---

12. Step 7 — OpenAPI / Swagger Documentation

Every professional API requires documentation. You must create an OpenAPI 3.0 specification file that documents all API Gateway endpoints. This is a YAML or JSON file that describes routes, request bodies, parameters, and responses.

12.1 What to Document

Your OpenAPI spec must include meaningful descriptions for all of the following:

- GET /api/movies — List all movies; document the optional ?title query parameter
- POST /api/movies — Create a movie; document the request body schema with title and description
- DELETE /api/movies — Delete all movies
- GET /api/movies/{id} — Get movie by ID; document the path parameter
- PUT /api/movies/{id} — Update movie; document path parameter and request body
- DELETE /api/movies/{id} — Delete movie by ID
- POST /api/billing — Create an order; document the request body with user_id, number_of_items, total_amount

12.2 Recommended Approach: SwaggerHub

SwaggerHub (swaggerhub.com) provides a free online editor for writing and validating OpenAPI specs. Steps:

63. Create a free account at swaggerhub.com
64. Click 'Create New API' and select OpenAPI 3.0
65. Use the editor to write your spec. The editor validates in real time and provides a visual preview of your documentation
66. Export the spec as openapi.yaml and save it to the root of your project

12.3 Minimum Required YAML Structure

Your spec file must at minimum contain:

```
openapi: 3.0.0
info:
  title: CRUD Master API Gateway
  version: 1.0.0
  description: API Gateway for movie inventory and billing microservices
paths:
  /api/movies:
    get:
      summary: Retrieve all movies
      # ... parameters, responses
    post:
      summary: Create a new movie
      # ... requestBody, responses
  # ... remaining endpoints
```

---

13. Step 8 — Postman Testing

Postman is used to test all API endpoints manually and to create an exportable collection that auditors can import and run. You must create tests for every endpoint and export the collection file to your repository.

13.1 Setting Up the Postman Collection

67. Open Postman and click 'New Collection'. Name it 'CRUD Master'.
68. Add a collection-level variable named GATEWAY_URL with value http://{{GATEWAY_IP}}:{{GATEWAY_PORT}} (or use the actual VM IP directly, e.g., http://192.168.56.10:3000).
69. Create requests for each endpoint inside the collection.

13.2 Required Requests to Create

Create and test these requests, setting Content-Type: application/json for all POST/PUT requests:

Method	Endpoint	Description
POST	`/api/movies`	Body: {title, description}. Expected: 200 + created movie object
GET	`/api/movies`	No body. Expected: 200 + array of all movies
GET	`/api/movies?title=...`	Query param. Expected: 200 + filtered movies array
GET	`/api/movies/:id`	Path param. Expected: 200 + single movie object
PUT	`/api/movies/:id`	Body: {title, description}. Expected: 200 + updated movie
DELETE	`/api/movies/:id`	Path param. Expected: 200 + success message
DELETE	`/api/movies`	No body. Expected: 200 + success message
POST	`/api/billing`	Body: {user_id, number_of_items, total_amount}. Expected: 200

13.3 Exporting the Collection

After verifying all requests work correctly, export the collection:

70. Right-click the collection in Postman's sidebar and select 'Export'.
71. Choose 'Collection v2.1' format.

72. Save the exported JSON file inside your project repository (e.g., at the root or in a postman/ directory).
73. Commit the file to git.

 Note: *The exported JSON file allows the auditor to import your exact test suite into their own Postman instance by clicking File > Import and selecting the file.*

---

14. Audit Checklist & Verification

This section maps directly to the audit questions. Use it as a final verification checklist before submitting.

14.1 VM Configuration Checks

Run these commands from the project root on the host machine:

```
vagrant up        # Should boot all 3 VMs without errors
vagrant status    # Should show 3 VMs all in 'running' state
cat .env          # Should show all credentials — no empty values
```

- Confirm gateway-vm, inventory-vm, and billing-vm all appear as 'running'.
- Confirm .env contains credentials for both databases, RabbitMQ, ports, and IPs.
- Grep the source files to ensure no credentials appear in .js files: grep -r 'password' srcs/ should return no hardcoded strings.

14.2 Inventory API Endpoints Verification

With Postman, using the gateway IP and port (e.g., http://192.168.56.10:3000):

74. POST /api/movies with {"title": "A new movie", "description": "Very short description"} — expect 200.
75. GET /api/movies — expect 200 and the newly created movie in the response array.
76. Import the Postman collection file. Run all requests. All should return expected responses.

14.3 Inventory Database Verification

```
vagrant ssh inventory-vm
sudo -i -u postgres
psql
\l                    # movies database should appear in list
\c movies
```

```
TABLE movies;          # Should show rows matching your Postman requests
```

14.4 Billing API Endpoints Verification

POST /api/billing with {"user_id": "20", "number_of_items": "99", "total_amount": "250"} — expect 200.

```
vagrant ssh billing-vm
sudo pm2 stop billing_app
sudo pm2 list           # billing_app should show status: stopped
```

Then POST /api/billing with {"user_id": "22", "number_of_items": "10", "total_amount": "50"} — still expect 200 from gateway.

14.5 Billing Database Verification

```
vagrant ssh billing-vm
sudo -i -u postgres
psql
\l                    # orders database should appear
\c orders
TABLE orders;          # user_id=20 should be present, user_id=22 should
NOT
```

14.6 Queue Resilience Verification

```
sudo pm2 start billing_app   # Restart the billing service
# Wait 3-5 seconds
TABLE orders;                # user_id=22 should NOW appear
```

This confirms that RabbitMQ held the message while billing_app was down and delivered it upon reconnect.

---

15. Troubleshooting

VMs not starting or timing out

- Check VirtualBox is installed and running by opening the VirtualBox GUI.
- If on Apple Silicon Mac, VirtualBox is not supported. Use VMware Fusion with the vagrant-vmware-desktop plugin.
- Run vagrant up --debug 2>&1 | tee vagrant_debug.log to capture the full debug log.
- Try vagrant destroy -f and vagrant up to start fresh.

PostgreSQL connection errors

- Ensure the PostgreSQL service is running inside the VM: sudo systemctl status postgresql
- Check the user and database were created: sudo -i -u postgres psql -c '\l'
- Verify the password in .env exactly matches what was used to create the DB user.

RabbitMQ connection errors from Gateway

- The gateway VM must be able to reach billing-vm on port 5672. Test with: nc -zv 192.168.56.12 5672 from inside gateway-vm.
- Ensure the RabbitMQ user credentials in .env match what was configured in the setup script.
- Check RabbitMQ is running on billing-vm: sudo systemctl status rabbitmq-server

Messages not processing after billing_app restart

- Ensure the queue was declared with durable: true in BOTH the gateway publisher and the billing consumer.
- Ensure messages were sent with { persistent: true } in channel.sendToQueue().
- Check PM2 logs for errors: sudo pm2 logs billing_app

Postman returns Connection Refused

- Confirm the correct gateway VM IP is being used (check vagrant status and vm.network in Vagrantfile).
- Verify the gateway app is running: vagrant ssh gateway-vm, then sudo pm2 list.
- Ensure port 3000 is not blocked by a firewall inside the VM: sudo ufw status